CM3070 Preliminary Project Report
(Preliminary Report + Feature Prototype)

Disaster Warning and Community Alert Mobile Application for the Philippines entitled "Babala sa Sakuna"

Template: CM3050 Mobile Development, Template 10.1 Developing a Mobile App for Local Disaster Preparedness and Response

Word Count 5243

Table of Contents

CHAPTER 1 – INTRODUCTION

1.1 Project Overview …….. 1
1.2 Motivation and Problem Statement …….. 1
1.3 Project Template and Scope ……. 1-2
1.4 Aim and Objectives ……. 2
1.5 Structure of the Report …… 2
1.6 1.6 Project Timeline ……. 2

CHAPTER 2 - LITERATURE REVIEW

2.1 Disaster Risk and Communication in the Philippines …… 3
2.2 Existing Official Information Sources …… 3
2.3 Global Platforms and Tools …… 3
2.4 Community-Based and Academic Approaches …….. 4
2.5 Mobile Technologies and Usability Considerations …… 5
2.6 Summary of Literature Gaps …….. 6

CHAPTER 3 – DESIGN

3.1 Design Goals ……. 6
3.2 System Architecture (High Level) …….. 6
3.3 Functional Design ……. 7
3.4 User Flow …… 8
3.5 Interface Design …… 8
3.6 Component Architecture …….. 9
3.7 Technology Choices ……. 9
3.8 DEI and Accessibility Considerations …… 10

CHAPTER 4 - FEATURE PROTOTYPE

4.1 Prototype Purpose and Scope ……. 11
4.2 Implementation Overview ……. 11
4.3 Prototype Screens and Features……. 12
4.4 Evaluation Approach ……. 12
4.5 Initial Evaluation and Findings ……. 13
4.6 Planned Improvements ……. 13
4.7 Summary ……. 14
4.8 Prototype Video Compliance …… 15

REFERENCES

CHAPTER 1 - INTRODUCTION

1.1 Project Overview

The Philippines ranks as one of the globe's most hazard-exposed nations hit often by storms, flooding, quakes, slope failures, along with volcanic activity (World Risk Report, 2023). For each threat type, a separate agency handles surveillance and alerts like PAGASA, PHIVOLCS, or NDRRMC; yet they operate in isolation when sharing updates via online platforms, social networks, text alerts, plus press statements (PAGASA, 2023; PHIVOLCS, 2023; NDRRMC, 2023). Because coordination lacks integration, public response may suffer from mixed signals, slow comprehension of danger levels, uneven readiness across regions.

This study introduces a mobile app called **"Babala sa Sakuna"** developed for disaster alerts in the Philippines. Rather than combining multiple tools, it works as one centralized system focused on smartphones. Information comes from reliable agencies, then gets filtered into simple updates anyone can understand. Instead of complex layouts, warnings appear in straightforward visuals and messages. Besides alerts, users also gain access to readiness guides tailored to local risks. To support real-time awareness, residents may submit observations directly through the interface.

1.2 Motivation and Problem Statement

In rapidly changing crises like when a powerful typhoon hits or sudden flooding begins, people usually depend on both government alerts and unchecked posts from social networks. Studies in the Philippines show that scattered updates mixed with false rumors may delay proper reactions (Bernardo, 2019; Flores & Abalos, 2021). Often, individuals find it hard to know which alerts are up to date, where impacts are happening, or what specific steps to follow.

The core issue tackled here is the absence of a single, easy-to-use mobile app tailored to local needs, which brings together disaster-related data for people in the Philippines. Instead of depending on fragmented updates and unofficial sources, the solution introduces a unified platform where individuals can check real-time warnings, explore danger zones via maps, and get actionable safety tips, even without internet.

1.3 Project Template and Scope

This project applies Template 10.1 in the CM3050 Mobile Development module to build a mobile app aimed at improving community preparedness for emergencies, it targets real-time response during local disasters. While designed for practical use, the system supports users before, during, and after crisis events. The approach combines simple navigation with key alert features, helping individuals act quickly when needed the most.

For now, the focus stays on building a working front-end model with fake data this means made-up alerts and test danger zones. Instead of full features, it allows simple movement between main pages like the overview screen, warning info, risk map, task list, and report section. Besides that, there's a part designed to work without internet, giving critical help

when signals are weak or missing. Alongside, early tests show how the system acts and whether users find it manageable.

Integration with live government alerts isn't part of this version; instead, the setup allows room for it down the line. Although direct data sharing now isn't active, the framework supports linking to public alert sources later on - when expanded in future updates.

1.4 Aim and Objectives

The main goal is designing a mobile app prototype tool that focuses on multiple hazards, helping Filipino groups grasp risks and act accordingly when disasters threaten.

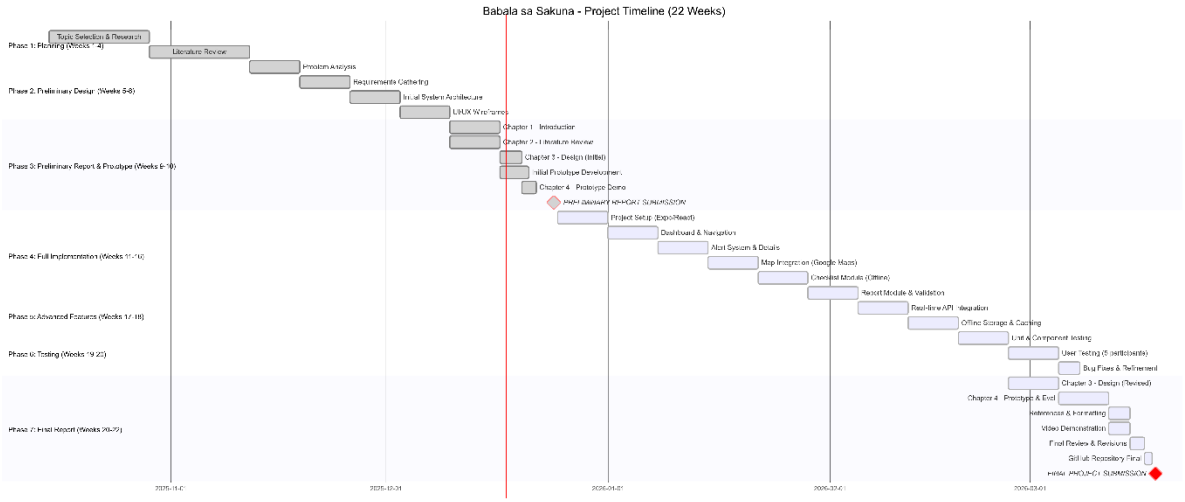
To reach this goal, the project sets these aims:

- To examine current disaster information platforms while spotting weaknesses in reach, ease of use, and regional adaptation.
- To define operational plus technical needs for a mobile app warning about multiple hazards.
- To build a system layout along with an interface enabling instant warnings, risk zone maps, checklists usable without internet, also allowing local updates.
- To build a test version showing key features especially the warning panel plus access to risk details.
- To assess the feature prototype through a suitable method focused on users.

1.5 Structure of the Report

The remainder of this early report includes three more sections. Chapter 2 covers research on disaster information systems and related scholarly work. Chapter 3 outlines the proposed system design, including the overall structure, user flows, and initial screen concepts. Chapter 4 describes the prototype implementation, explains how the working model was developed, summarises initial findings, and identifies potential improvements for future iterations.

1.6 Project Timeline



CHAPTER 2 - LITERATURE REVIEW

2.1 Disaster Risk and Communication in the Philippines

The Philippines often ranks high in global disaster risk because it faces frequent natural hazards along with social and economic weaknesses (World Risk Report, 2023; UNDRR, 2022). Over twenty typhoons move into the nation's monitoring zone annually (PAGASA, 2023); meanwhile, its location on the Pacific Ring of Fire increases threats from earthquakes and eruptions (PHIVOLCS, 2023). Because of this, clear and fast communication during emergencies becomes essential for public safety.

Research on disaster messaging in the Philippines points to challenges like dependence on conventional outlets, inconsistent digital access, alongside widespread unchecked content online (Bernardo, 2019; Flores & Abalos, 2021). Despite rising internet and phone usage (Santos, 2018), incorporating tech into structured alert networks is still limited.

2.2 Existing Official Information Sources

Multiple groups in the Philippines currently share info on disasters

- PAGASA handles weather predictions, storm updates, or flood alerts (PAGASA, 2023).
- PHIVOLCS releases alerts about quakes, volcanic activity, or tidal waves (PHIVOLCS, 2023).
- NDRRMC manages nationwide emergency actions; it also shares verified updates and status summaries (NDRRMC, 2027).

Every organization usually runs its own site while sharing alerts through Facebook updates, PDF files, or media statements. Because these sources are split up, people need to check several platforms at once to understand the full danger, something that can be hard or even impossible when a crisis unfolds quickly.

2.3 Global Platforms and Tools

Beyond government bodies, worldwide platforms like Google SOS Alerts deliver situation specific updates when major emergencies occur (Google, 2023). Such alerts appear through Search, Maps, or phone notifications, giving people general insights into nearby risks. Still, these global systems usually cover wide areas instead of precise local zones like individual barangays and often function separately from official national alert networks.

An additional key case is Project NOAH, initially launched by the Philippine state to merge various data types like rain measurements, river heights, or flood predictions into an interactive map system (Project NOAH, 2020). Despite its progress in displaying risk details more clearly, when authorities stopped funding it, this weakened the tool's long-term use alongside present emergency procedures.

2.4 Community-Based and Academic Approaches

Studies into local disaster alert networks highlight the value of grassroots insights alongside public involvement. Flores and Abalos (2021) show that hazard maps made by residents, combined with area-specific updates, may strengthen state-issued alerts particularly in isolated or under-resourced regions. Still, blending citizen inputs with government records needs thoughtful planning so misinformation isn't accidentally shared.

Bernardo (2019) points out difficulties in sharing risk details with diverse groups, since confusing terms and uneven messaging styles can lead to errors. As a result, fresh tools should focus on clear, straightforward, uniform design particularly within smartphone apps.

2.5 Mobile Technologies and Usability Considerations

Mobile phones are now the main way people get info in the Philippines, especially using smartphones (Santos, 2018). Because of this shift, apps can help share disaster alerts yet they come with usability hurdles. Under high pressure, users may struggle to focus; meanwhile, poor signal or low power adds difficulty. Moreover, designs need to work for those who read less well or lack tech experience.

Human-computer interaction studies show that simple layouts, easy-to-follow menus, or reduced mental effort matter most in crisis situations. Tools like step-by-step lists, symbol-driven interfaces, yet location-based displays may speed up threat recognition provided they're built thoughtfully. Functionality without internet access is likewise advised when connectivity fails amid emergencies.

2.6 Critical Evaluation of Existing Systems

The current systems mentioned earlier show notable advantages they're credible, frequently include detailed data, while also advancing gradually due to refined models and tracking methods (PAGESA, 2023; PHIVOLCS, 2023; NDRRMC, 2023; UNDRR, 2022). Still, various ongoing weaknesses matter for this study.

- Disjointed data sources split details across platforms, so people must piece together insights themselves yet no unified view exists to simplify understanding because systems operate in isolation rather than sync efforts.
- Some global tools lack precise location data local info like neighbourhoods might be missing. National platforms often have layouts hard to understand for average people.
- Usability problems arise because numerous government sites lack mobile optimization; meanwhile, PDF updates prove hard to navigate on smartphones.
- Lack of offline functionality: many platforms depend on internet access even though connections often fail when crises occur.
- Poor inclusion of local views: formal setups rarely gather organised feedback from impacted individuals.

Such challenges highlight the need for a single app focused on ease of use, broad hazard alerts, plus location-specific info. This initiative takes advantage of current platforms especially

their trusted sources but improves them by prioritising mobile access, real-user needs, offline tools, along with simple ways for communities to share updates.

2.7 Summary of Literature Gaps

In short, existing studies along with real world applications suggest:

- The Philippines has solid systems in place to track natural hazards.
- Interaction and how users feel about it stand out as less effective links in the process.
- App-based tools can work due to widespread mobile use though thoughtful planning and testing matter.

The suggested project addresses these shortcomings through attention to interface layout, combining data sources, readiness tools without internet access using assessment methods based on human-computer interaction ideas.

CHAPTER 3 - DESIGN

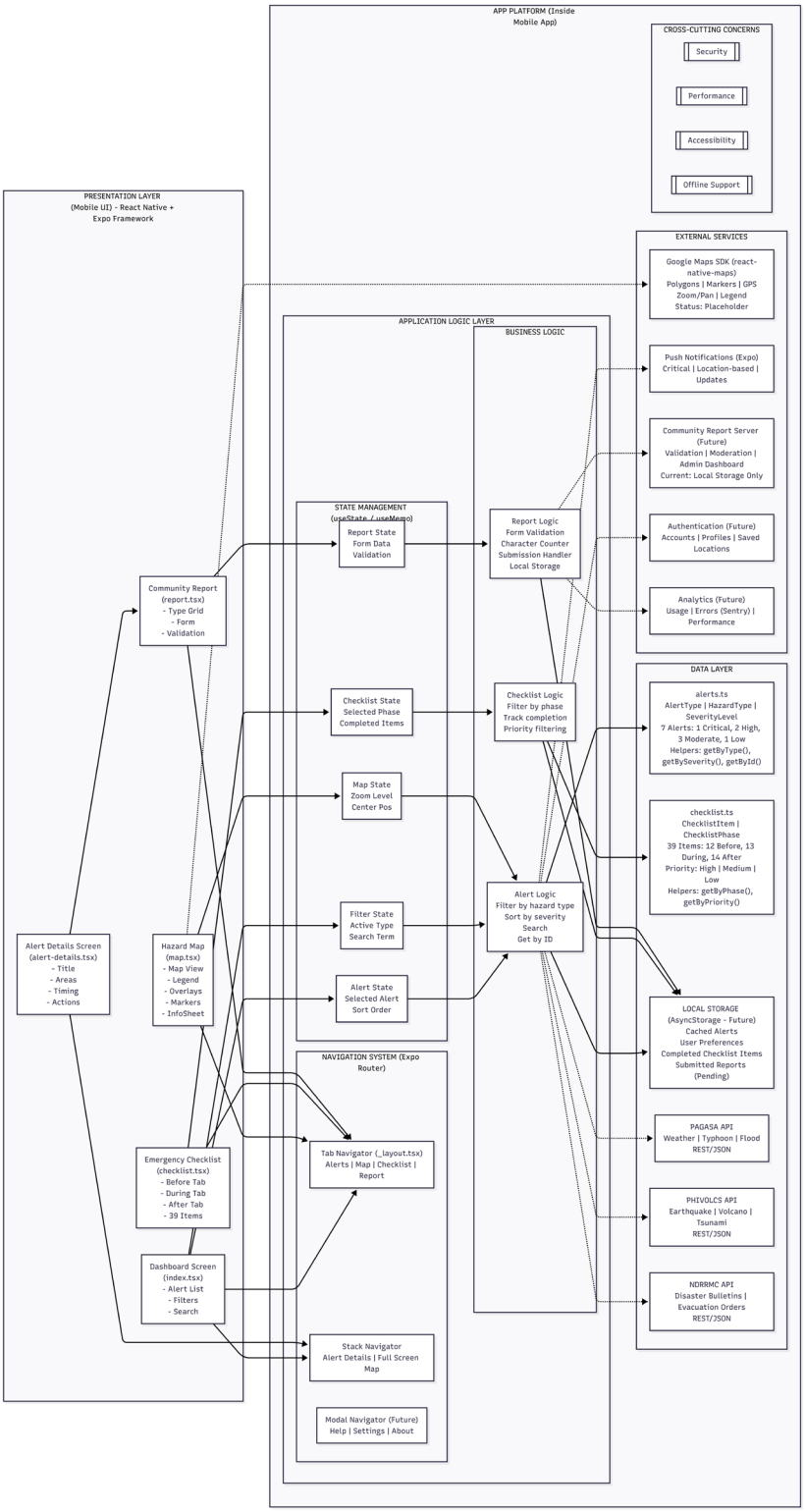
3.1 Design Goals

From prior research plus an examination of existing issues, the development of the alert system follows these objectives:

- Crisis situations need simple messages, using common words helps prevent misunderstandings, because complicated phrasing increases mistakes, according to Bernardo (2019).
- Fast understanding helps individuals recognize what's happening along with required steps quickly, fitting human to machine communication standards during emergencies.
- Emphasize local conditions by pointing out specific locations and affected areas, tackling gaps in detailed barangay level information noted in earlier research.
- Resilience: Provide key guidance even when signals are weak, Santos (2018) found that numerous Filipinos struggle with connectivity during crises.
- Adaptability: Allows future connections to real-time data sources via a scalable structure, using modular components that support system integration when needed.

3.2 System Architecture (High Level)

The setup is thought of as a flexible phone app featuring these main parts:



Caption: Full system setup, displaying every layer, part, how data moves, along with possible expansion spots

Key Architecture Decisions:

- Layered design distinct components work separately; this allows teams to build and check parts on their own.
- File-Based Routing Expo Router for intuitive navigation that mirrors the file structure
- Component-Based UI Reusable, maintainable components following React Native best practices.
- Type-Safe uses TypeScript to catch errors during compilation instead of runtime
- Offline-first design uses local storage to support key functions when internet is weak or missing (Santos, 2018).
- Extensible straightforward to incorporate actual API connections later on.

The structure includes five key levels:

1. Presentation Layer (Mobile UI)

Builds all screen users see like dashboards, alerts, maps, checklists, also reports. Primarily built for Android, though adaptable to other systems using React Native.

2. Application Logic Layer

Handles navigation alongside app state like active alerts or filter options while managing operations including sorting warnings based on category or seriousness using conditional checks instead of direct commands. Checks simple rules for community submissions, like missing entries.

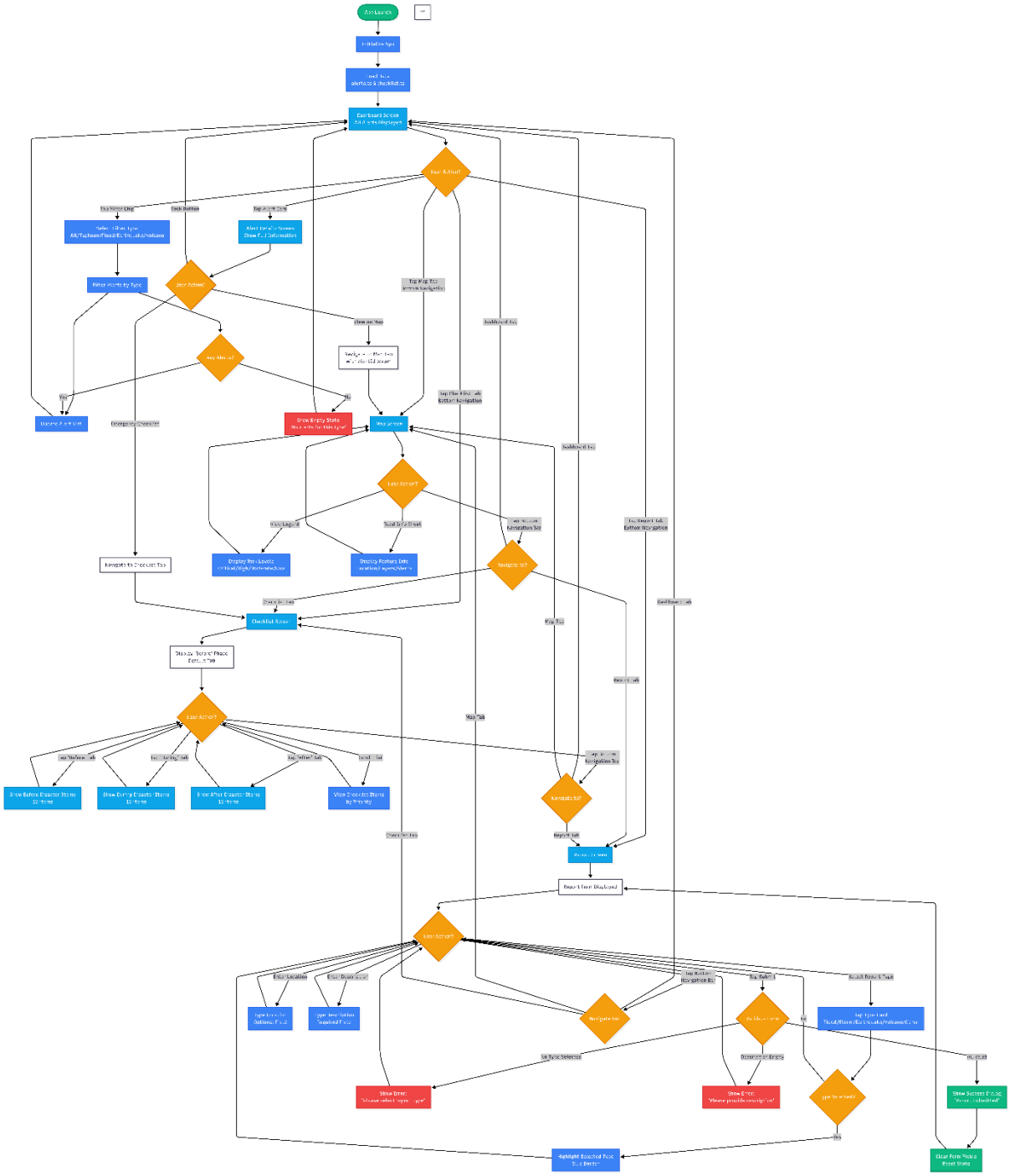
3. Data Layer (Prototype)

In this version, the system relies on fixed JSON files kept offline to show warnings, danger areas, and task lists though these are not pulled from external sources. Instead, they're embedded directly into the interface for quick access while limiting dynamic updates during use. In future versions, this module could link to online data sources from PAGASA, also pulling inputs from PHIVOLCS, while integrating feeds from NDRRMC.

4. Mapping Service Integration

Uses Google Maps SDK to show danger areas; also displays where the user is (Google, 2023). In the early model stage, risky areas appear through added visual tags.

5. Local Storage / Offline Support
Keeps the emergency checklist along with saved warnings directly on the phone through Async Storage. Keeps key advice available without internet connection.



Caption: Full overview of user steps, highlighting navigation options, choices, or interactions at each stage

This setup keeps tasks distinct while allowing gradual improvements: the interface might first use fake data then switch to real APIs without major changes.

3.3 Functional Design

The main functions of the system show up in its primary components:

1. **Alert Module**

- Shows active warnings including kind, seriousness, brief description, also when last changed.
- Let's users filter based on danger type such as storm, flooding, quake, or eruption.
- Let's users access specific details about every alert through direct links.

2. **Hazard Map Module**

- Shows risky zones using colored layers. Red means severe, orange stands for high, amber indicates medium, while blue shows minor risks.
- Shows roughly where the user is, if allowed by settings.
- Links to certain warnings, showing impacted areas using circular zones.

3. **Checklist Module**

- Offers organized emergency lists with 39 practical steps, designed for use without internet access.
- Split into three stages: "Before", followed by "During", then ending with "After" an occurrence.
- Enables fast checking of suggested steps by their importance order.

4. **Reporting Module**

- Users can send basic reports about local issues by choosing a category like flooded areas or obstructed streets and including a brief note.
- In the first model, data files stay on local devices.
- In future updates, reports might go to a central server where local authorities would then check and combine them using secure channels.

5. **Navigation Framework**

- Enables navigation from dashboard to map, then checklist; continues to report before reaching detailed screens.
- Uses a bottom bar to switch between key screens, while moving deeper into sections through sequential steps.

3.4 User Flow

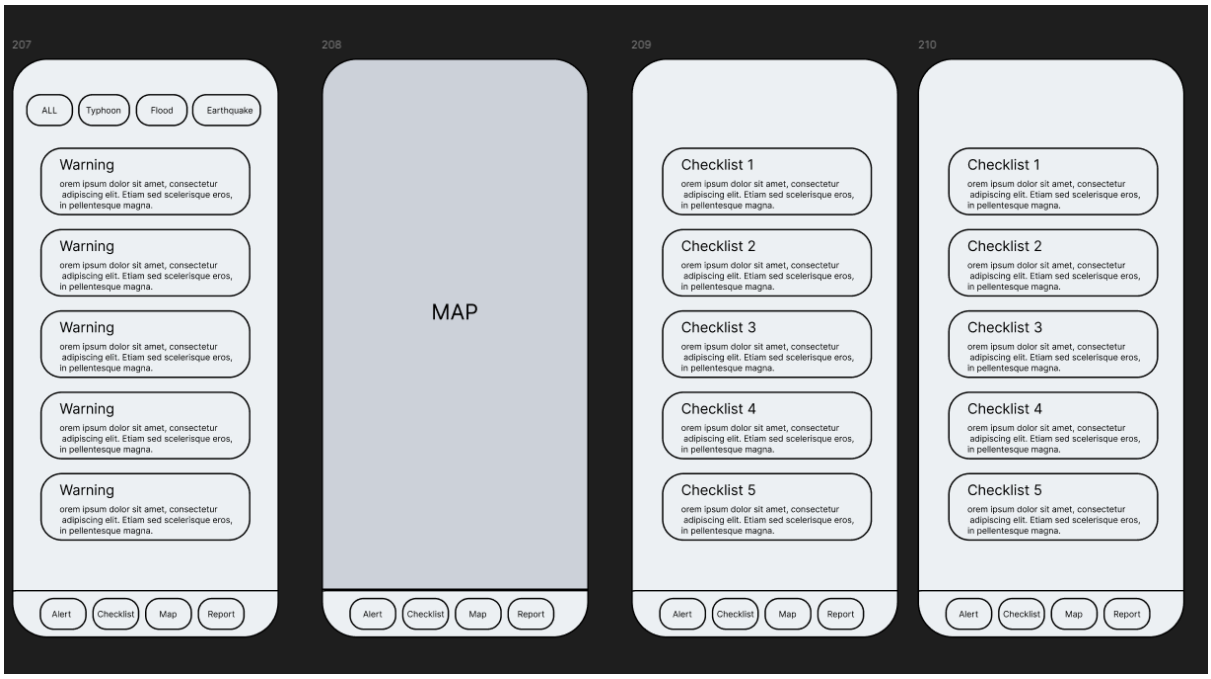
The primary path users follow aims to stay straightforward, meeting targets around quickness plus clear communication:

1. App starts the screen shows current warnings ordered by how serious they are.
2. User checks the list, then picks an alert.
3. The Alert details window displays impacted zones along with when they occurred, followed by suggested steps to take.
4. After that, users may:
    a. access the Hazard Map to see where risks are located, also how severe they might be,
    b. use the Checklist to find what steps to take next.
5. The user may change tabs at any time through the lower navigation panel.

This sequence matches the main concerns people usually face in emergencies what's going on, where it is occurring, how it impacts them, and what actions to take.

3.5 Interface Design

The interface layout follows mobile usability guidelines along with limits found in past studies (Flores and Abalos, 2021; Santos, 2018). To lower mental effort under pressure, strong colour contrasts combined with few visual extras were used backed by Bernardo (2019).



Caption: Layouts for the app's four key screens

1. **Dashboard Screen**
   - A clear header displays the app's name "Babala sa Sakuna" alongside a subheading that tracks how many alerts are active.
   - Sort chips by danger type (All, Typhoon, Flood, Earthquake, Volcano).
   - Warning labels showing danger symbol, heading, risk level tag along with time stamp.

2. **Alert Details Screen**
   - A straightforward hazard name with a clear risk level appears first. The label uses simple terms to show how serious it is.
   - Descriptions, along with regions impacted, time frames, also guidance on actions to take.
   - Action buttons include 'View on Map' along with 'Open Checklist'.

3. Hazard Map Screen:
   - Dynamic map display using Google Maps technology.

- Visual cues showing warning spots using colour intensity to reflect seriousness.
- Circles drawn around points to show impact zones.
- A key showing how colors represent different risk levels.
- A popup window showing the chosen warning. It gives a brief overview using simple terms.

4. **Checklist Screen:**
   - A tab-based design shows the stages: Before, During, also After.
   - Prioritized entries marked by color: red stands for high, amber shows medium, green indicates low.
   - Clear descriptions for each action item.

5. **Reporting Screen**
   - A basic form featuring a grid to pick hazard types.
   - Location is optional.
   - A required description box includes a count of characters maximum 500.
   - Rules for sending work plus responses after review.

Screen/ Component	File Path	Purpose	Components Used
SCREENS			
Dashboard	app/(tabs)/index.tsx	Main alert feed with filters. Shows all active alerts sorted by severity. Primary entry point for users. Filterable by hazard type (typhoon/flood/etc).	• AlertCard • FilterChips • SeverityBadge • FlatList (RN) • TouchableOpacity (RN) • Lucide icons
Alert Details	app/alert-details.tsx	Detailed view of single alert. Shows description, affected areas, timing, recommendations. Includes CTAs to map/checklist. Accessed via alert card press.	• SeverityBadge • ScrollView (RN) • TouchableOpacity (RN) • Lucide icons
Hazard Map	app/(tabs)/map.tsx	Visual hazard map (placeholder). Shows colored regions indicating risk levels. Includes legend and info sheet explaining features. Future: Google Maps integration.	• View (RN) • ScrollView (RN) • Lucide icons • Future: MapView
Checklist	app/(tabs)/checklist.tsx	Emergency preparedness guide. 3 phases: Before/During/After disaster. 30 actionable items organized by priority and phase.	• ChecklistItemCard • FlatList (RN) • TouchableOpacity (RN) • Lucide icons
Community Report	app/(tabs)/report.tsx	User-submitted hazard reports. Form with type selector, optional location, required description. Includes submission validation and success dialog. Community engagement feature.	• TextInput (RN) • TouchableOpacity (RN) • ScrollView (RN) • Alert (RN) • Lucide icons
COMPONENTS			
AlertCard	components/alert-card.tsx	Reusable alert display card. Shows title, type, severity, timestamp, description (2 lines), affected areas (first 2), "more" indicator, arrows.	• SeverityBadge • TouchableOpacity (RN) • View (RN) • Text (RN) • Lucide icons
Severity Badge	components/alert-card.tsx (sub-component)	Color coded severity indicator. Shows severity level with appropriate color (Critical=Red, High=Orange, Moderate=Amber, Low=Blue). Props: severity.	• View (RN) • Text (RN)
FilterChips	components/filter-chips.tsx	Horizontal scrollable filters. Allows filtering by hazard type. Visual active state (blue bg). Props: selectedFilter, onFilterChange	• ScrollView (RN) • TouchableOpacity (RN) • View (RN) • Text (RN) • Lucide icons
ChecklistItemCard	components/checklist-item-card.tsx	Checklist item display card. Shows priority indicator (colored left bar), category, title, description. Props: item. Priority: high/medium/low.	• View (RN) • Text (RN) • Lucide icons
NAVIGATION			
Tab Layout	app/(tabs)/_layout.tsx	Bottom tab navigation config. 4 tabs: Dashboard, Map, Checklist, Report. Includes icon and label for each tab. Active tint color: #0E4DF3 (blue-600).	• Tabs (Expo Router) • TabBarIcon • Lucide icons
Root Layout	app/_layout.tsx	App-level navigation setup. Wraps tab navigation. Handles stack screens (alert-details, modal). Loads fonts and provides.	• Stack (Expo Router) • FonsProvider • SafeAreaProvider
DATA LAYER			
Alerts Data	data/alerts.ts	Mock disaster alert data. 7 sample alerts (4 hazard types). Helper functions: • getAlertsBySeverity() • getAlertsByType() • getAlertById()	N/A - Pure TypeScript
Checklist Data	data/checklist.ts	Emergency checklist data. 30 items across 3 phases. Helper function: getChecklistByPhase() Fields: id, phase, category, title, description, priority.	N/A - Pure TypeScript

LEGEND:
- RN = React Native built-in component
- Expo Router = file-based navigation from Expo
- lucide-icons = icon library from @expo/vector-icons

Color Category	Color	Hex Code	Usage Context
SEVERITY LEVELS			
Critical	Red-600	#DC2626	• Critical severity badges • High priority indicators • Emergency alerts • Danger zone markers • Used in: AlertCard, SeverityBadge, Map Legend
High	Orange-500	#EA580C	• High severity badges • Warning states • Urgent notifications • Used in: SeverityBadge, Map Legend
Moderate	Amber-500	#F59E0B	• Moderate severity badges • Caution indicators • Medium priority checklist items • Used in: SeverityBadge, ChecklistItemCard
Low	Blue-500	3B82F6	• Low severity badges • Information states • Advisory alerts • Used in: SeverityBadge
PRIORITY LEVELS (Checklist)			
High Priority	Red-600	#DC2626	• High priority checklist items • Left border indicator • Icon color • Used in: ChecklistItemCard
Medium Priority	Amber-500	#F59E0B	• Medium priority checklist items • Left border indicator • Icon color • Used in: ChecklistItemCard
Low Priority	Green-500	5D9B81	• Low priority checklist items • Left border indicator • Icon color • Success states • Used in: ChecklistItemCard, Success dialogs
UI ELEMENTS			
Primary Action	Blue-600	0066A5A9	• Primary buttons (Submit, View Map) • Active filter chips • Active tab link color • Links and CTAs • Used in: All screens, FilterChips, Tab Nav
Primary Dark	Blue-6250	4085C7	• Button pressed state • Hover states (web) • Darker accent elements
Background	Gray-50	#F9FAFB	• App background • Screen background • Used in: All screens
Card Background	White	#FFFFFF	• AlertCard background • ChecklistItemCard background • Form input background • Modal background
Border	Gray-200	#E5E7EB	• Card borders • Input borders • Dividers • Section separators
Text Primary	Gray-1110270	900	• Headings • Alert titles • Primary content text • Used in: All text elements
Text Secondary	Gray-40	6080563	• Descriptions • Timestamps • Subtitles • Helper text
Text Tertiary	Gray-54	4B8CA3AF	• Placeholders • Disabled text • Inactive tabs • Footnotes
Error	Red-500	#EF4444	• Form validation errors • Error messages • Alert indicators • Used in: Report form validation
Success	Green-104	5D9B81	• Success messages • Confirmation dialogs • Positive feedback • Used in: Report submission success
Shadow	Black (#10 alpha)	000000 with alpha	• Card shadows (opacity: 0.1) • Elevation effects • Drop shadows

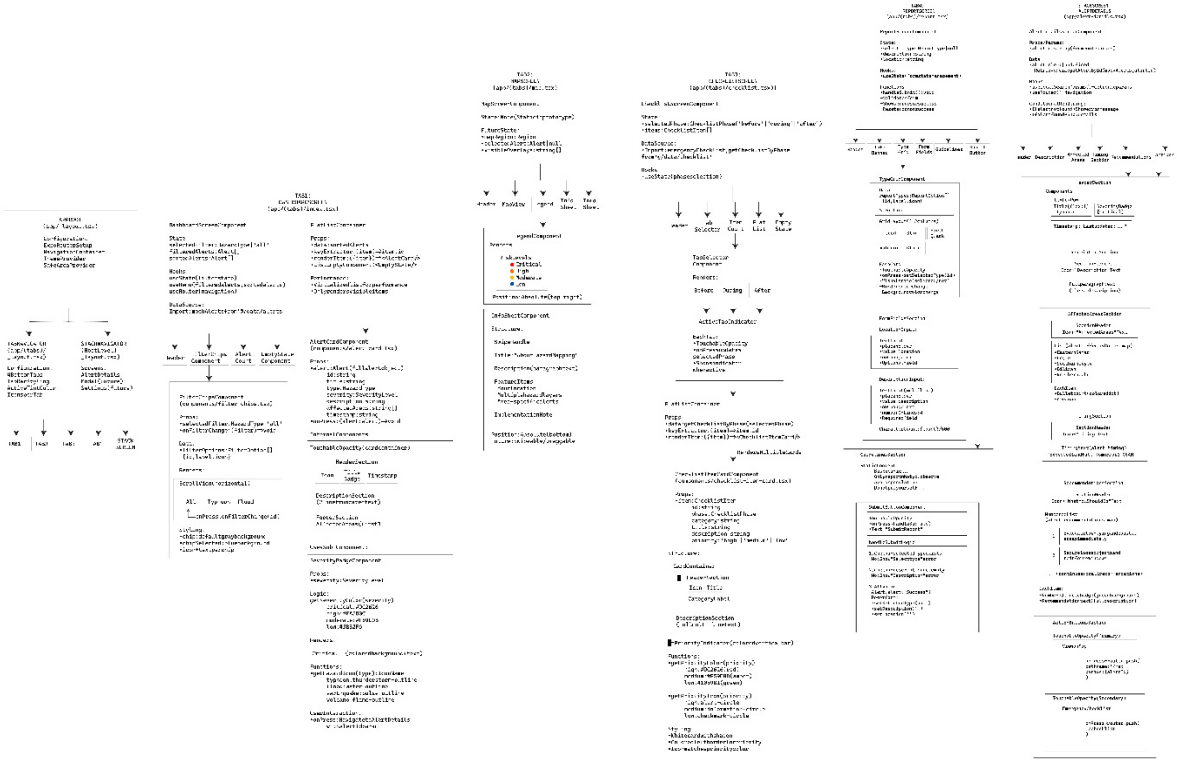
DESIGN SYSTEM NOTES:
- Color palette follows TailwindCSS naming convention for consistency
- All colors have sufficient contrast ratios for WCAG AA compliance
- Severity colors align with universal danger signaling (red=danger, amber=caution, blue=info)
- Primary color (#0066A5A9 - Sky Blue) chosen for Philippines water/ocean association
- Neutral grays used extensively for clean, professional appearance

Caption: Colour scheme using meaning-based hues to indicate seriousness levels alongside interface component tones

The colour scheme follows widely accepted alert codes: red means critical, orange indicates high risk, amber stands for medium level, whereas blue covers low or advisory cases. It matches current warning formats; therefore, users learn it faster.

3.6 Component Architecture

The app relies on a modular, component-driven structure featuring 15 separate interface elements



Caption: Structure of components displaying how they’re connected, along with data passed from top to bottom

Reusable Components (4):

1. **Alert Card:** Shows key alert details including a status indicator, commonly found on Dashboards
2. **A color-based tag** showing how serious something appears on various pages.
3. **Filter Chips:** Horizontal scrollable filter buttons, used in Dashboard
4. **Checklist Item Card:** Priority-coded checklist item, used in Checklist screen

Component Design Principles:

1. Single Responsibility. Each component has one clear purpose
2. Props-Based Configuration Behaviour controlled via props for flexibility
3. Type Safety All props typed with TypeScript interfaces
4. Reusable elements recurring structures moved into unified modules
5. Efficiency Flat List enables smooth scrolling through large data sets by rendering only visible items
6. Composition Complex UIs built from simple, testable components

3.7 Technology Choices

While the prototype works without a complete backend, its structure expects one using components like; though not yet active

- React Native (Expo SDK 54) for cross-platform mobile development
- Expo Router for file-based navigation
- Google Maps SDK (react-native-maps) for hazard visualisation
- Local storage handles checklists when offline while keeping data ready to use later through temporary caching mechanisms.
- TypeScript enables safer coding through static typing
- RESTful API clients in a future phase for connecting to PAGASA, PHIVOLCS and NDRRMC

These tools are commonly applied, clearly explained, besides fitting the specific challenge area offering a realistic path to building a working system.

3.8 DEI and Accessibility Considerations

Diversity, equity, and inclusion shape the system's structure helping people regardless of reading ability, age, or internet quality. High-contrast visuals along with straightforward warning cues cut confusion during urgent moments. Alerts appear in simple terms, avoiding technical jargon. Critical steps and safety advice work without Wi-Fi, targeting areas where mobile networks fail often. Future updates will offer content in multiple languages, such as English plus Tagalog, increasing reach across linguistic groups. Button sizes and layout spacing accommodate small displays and those with limited hand mobility. Together, these decisions target gaps tied to hardware limits, mental effort, and spoken language differences.

CHAPTER 4 - FEATURE PROTOTYPE

4.1 Prototype Purpose and Scope

The initial version highlights how users interact with the emergency alert system, specifically:

- The alert dashboard includes filters that help narrow down information.
- Navigation to alert details with comprehensive information.
- Move on to the interactive hazard map using a click or tap.
- Use the checklist without internet connection.
- A simple tool for community reports, including checks to confirm accuracy.

The goal is showing the suggested setup works technically also that interactions feel natural, despite relying on mock data during this early stage.

4.2 Implementation Overview

The prototype used React Native alongside Expo SDK 54, matching the plans from Chapter 3. It includes:

- A bottom-tab navigator with routes for Dashboard, Map, Checklist and Report
- A stack-based system moves users from the main screen to specific alerts using transitions that maintain context flow while enabling return paths through hierarchical ordering
- Reusable parts for filter chips, alert cards also severity indicators
- Structure handling data tasks using services like AlertService or CacheService
- Combining expo-location to handle user location tracking through device sensors instead of manual input
- Storing data locally when offline using AsyncStorage

Because alert data uses TypeScript objects that are strictly typed, every alert includes an ID, a title, plus details like description. Each one specifies its category such as typhoon or flood or even earthquake and volcano. Severity ranges from critical down to low, influencing how the system displays them. Affected regions are listed alongside time-related info along with suggested actions. On the dashboard, alerts get sorted based on seriousness instead of just kind. Filters help narrow results either by risk level or event type. Once someone picks an alert, its unique ID moves forward through Expo Router's parameter setup. That key then loads full content inside the detail view.

The hazard map display combines Google Maps SDK along with markers showing warning spots, shaded according to risk level. Around each threat, circular zones illustrate the impacted region. At the base, a sliding panel presents specifics of the chosen alert, including route tools.

The checklist interface includes 39 entries divided across three stages (Before: 12, During: 13, After: 14). Data is saved on-device so it works without internet. On the report page, users fill out a verified template choosing a category, adding a place if needed, along with a mandatory explanation.

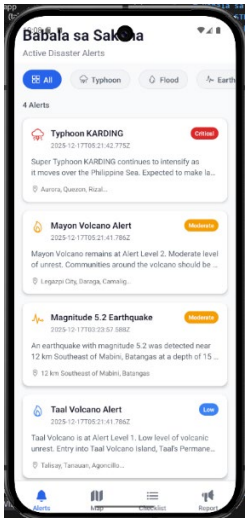
4.3 Prototype Screens and Features

The prototype includes these features:

Alert Dashboard:

Shows a movable lineup of warning boxes ordered by seriousness (most urgent on top)
Allows selection based on disaster category (options include "All", "Typhoon", "Flood", "Earthquake", or "Volcano")

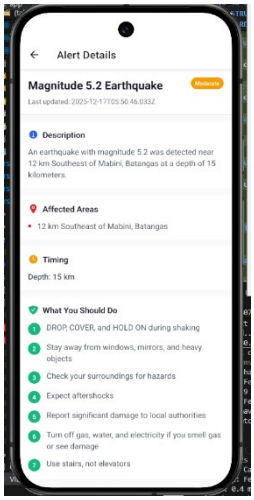
Pull-down action to reload notifications
Loading indicator during data fetch
Lets users select a card to access the Alert Details view



Caption: View of dashboard displaying current alerts, using filter tags along with level indicators

Alert Details Screen:

Show the alert title; include a severity indicator along with the most recent update time
Shows complete details, lists impacted zones along with schedule data
Lists clear steps based on the kind of risk involved
Features interactive options like “View on Map” or “Open Checklist”



Caption: Alert Details screen showing affected areas, timing, and recommendations

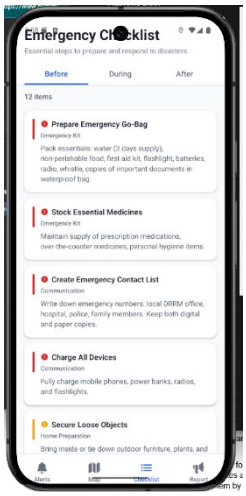
Hazard Map Screen:

Interactive Google Maps view centred on the Philippines
Coloured markers show every ongoing alert, with hues based on how serious they are
Circular zones indicating estimated impact regions
A user's location appears on screen if access is allowed
A 'My Location' button lets users snap back to their current spot
A bottom sheet showing a chosen alert overview
A key showing what each colour means

Caption: Live hazard map showing warning symbols along with zones marked by impact rings

Emergency Checklist Screen:

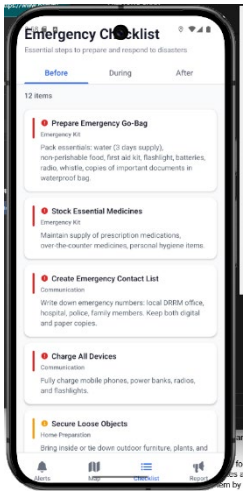
Tab navigation for Before/During/After phases
39 total items with priority indicators (high, medium, low)
A single entry displays its type, name along with a full explanation
Offline access provided by design



Caption: Emergency Checklist with phase tabs and priority-coded items

Reporting Screen:

A layout showing different report types, each with a symbol beside it
A location entry box is available if needed
Mandatory description section including a count
Reporting standards were shown
Checking forms along with alerts for mistakes
A message appears after submitting to confirm success



Caption: Form for submitting reports, showing validation messages

Together, these parts show how the main interactions work proving the architecture and design ideas fit together in a real interface. While each piece functions separately, their combination supports the overall structure effectively.

4.4 Evaluation Approach

This project emphasizes how users interact with the system, not just technical efficiency; therefore, assessment draws from human-computer interaction techniques. Instead of measuring speed or accuracy alone, it uses practical observation and feedback sessions

Heuristic Evaluation

The layout was checked using Nielsen's 10 usability rules; for instance, system feedback like loading signs or severity tags improves clarity. Regarding alignment with everyday experience, known hazard symbols were used instead of abstract ones. Users can navigate back or switch tabs easily this supports autonomy during tasks. Colour use stays uniform across screens, which aids recognition over time. To reduce mistakes, forms include live validation before submission.

Task-Based User Testing (Small-Scale)

A handful of selected participants local Filipino students and relatives aware of regional disaster situations were invited to carry out defined activities

1. Find the worst active warning
2. Go to the hazard map then explain where danger might occur
3. Find guidance that explains the steps to take
4. Check the list to identify actions needed prior to a disaster
5. Send in an easy update from the local area

During these sessions, the observer noted where participants hesitated, misunderstood or struggled with navigation.

Post-Session Feedback

Respondents answered broad questions on aspects they saw as understandable, unclear, comforting, or absent.

4.5 Initial Evaluation and Findings

First tests involving few participants (n=5) revealed these results:

Strengths:

1. The dashboard-style interface, along with filter chips, was called "simple to grasp" by 4 out of 5 users.
2. Participants liked having severity indicators along with time stamps displayed on alert cards.
3. The colour-based alert levels became clear quickly through short use.
4. The combination of notifications, maps, or checklists within one application felt helpful also calming to users
5. Offline checklists mattered a lot particularly for those hit by network problems in earlier storms.

6. Task completion stood at a strong level; four out of five participants finished all assignments independently

Weaknesses and Areas for Improvement:

1. Two out of five participants were unsure at first about what the colour-coded severity labels meant; therefore, providing a brief guide during initial access could help. Because clarity improved after exposure, early guidance may support quicker understanding. Since no one figured it instantly, integrating a simple explanation upon entry might enhance usability. While most caught on over time, immediate comprehension would likely rise with contextual cues present from the start.
2. The map needed an active Google Maps API key to work properly. When tested using a dummy version, people found it provided fewer details than anticipated.
3. The reporting function's goal wasn't clearly explained. Details like recipient roles or post-submission steps were missing.
4. A single user proposed bigger tap areas for filter chips when using compact displays.

Results match earlier work highlighting precise messaging about risks especially reducing uncertainty (Bernardo, 2019); similar outcomes appear in studies by Flores and Abalos (2021).

4.6 Planned Improvements

Given the first review alongside intended goals, upcoming versions will include these upgrades:

1. **Improved map links**
   Full use of Google Maps API featuring live updates on one risk kind like quake details from PHIVOLCS.
2. **Severity Guide and Details**
   Include a startup screen or pop-up guide showing what each level means, using colours and symbols helpful for new users. While launching the app, display brief explanations through tooltips or an intro panel to clarify ratings. Instead of assuming familiarity, present clear examples matching icons to their meaning early on. This supports understanding without relying on prior knowledge.
3. **Enhanced reporting clarity**
   Show a short note on what happens to community feedback, while including progress updates for each submission.
4. **Enhanced visual layout**
   Enlarge tap areas for filter tags while adjusting gaps this boosts usability on compact screens. Layout changes help users interact more easily, especially where space is limited. Bigger buttons plus clearer separation make navigation simpler for everyone.
5. **Use Expo Notifications** to send key alerts if the app isn't open. These messages pop up instantly, even when users aren't actively using it.
6. **Multilingual Option**
   Include Tagalog to reach more users this improves access across different groups while expanding usability naturally.

4.7 Summary

The feature prototype shows the main idea single app for disaster warnings and local alerts in the Philippines can work in real-world conditions while meeting basic usability standards. Early testing with five participants indicates people grasp how alerts, maps, and checklists fit together most completed tasks successfully (80%) without help. However, some parts need

improvement: users struggled at first with the color-coded labels, also the intent behind the report function wasn't fully clear.

Later studies will aim to improve live data handling in the prototype, finish integrating Google Maps, while also testing the system more thoroughly using a bigger group of users.

References

Bernardo, A. (2019) ‘Challenges in Philippine Disaster Communication’, Journal of Asian Public Policy, 12(3), pp. 312-329.

Flores, M. and Abalos, J. (2021) ‘Community-based disaster information systems in the Philippines: A review’, Philippine Journal of Community Development, 56(2), pp. 45-68.

Google (2023) Google SOS Alerts. Available at: https://www.google.org/crisisresponse/sosalerts (Accessed: 30 November 2025).

National Disaster Risk Reduction and Management Council (2023) NDRRMC Advisories and Bulletins. Available at: https://ndrrmc.gov.ph (Accessed: 30 November 2025).

PAGASA (2023) PAGASA Official Website. Available at: https://www.pagasa.dost.gov.ph (Accessed: 30 November 2025).

PHIVOLCS (2023) PHIVOLCS Earthquake Information. Available at: https://www.phivolcs.dost.gov.ph (Accessed: 30 November 2025).

Project NOAH (2020) Nationwide Operational Assessment of Hazards Website. Available at: https://noah.up.edu.ph (Accessed: 30 November 2025).

Santos, R. (2018) ‘Mobile readiness and digital access in Filipino households’, ICT Asia Journal, 7(1), pp. 20-34.

UNDRR (2022) Global Assessment Report on Disaster Risk Reduction. United Nations. Available at: https://www.undrr.org (Accessed: 30 November 2025).

World Risk Report (2023) World Risk Index 2023. Available at: https://www.worldriskreport.org (Accessed: 30 November 2025).

Technology References

React Native & Expo

Meta Platforms, Inc. (2024) React Native: Learn once, write anywhere. Available at: https://reactnative.dev (Accessed: 01 December 2025).

Expo (2024) Expo Documentation. Available at: https://docs.expo.dev (Accessed: 01 December 2025).

Expo (2024) Expo Router: File-based routing for React Native. Available at: https://docs.expo.dev/router/introduction (Accessed: 01 December 2025).

TypeScript

Microsoft (2024) TypeScript: JavaScript with syntax for types. Available at: https://www.typescriptlang.org (Accessed: 15 December 2025).

Mapping & Location

Google (2024) Google Maps Platform Documentation. Available at: https://developers.google.com/maps/documentation (Accessed: 15 December 2025).

Expo (2024) Expo Location: Access device location. Available at: https://docs.expo.dev/versions/latest/sdk/location (Accessed: 15 December 2025).

Airbnb (2024) react-native-maps: React Native Map components for iOS and Android. Available at: https://github.com/react-native-maps/react-native-maps (Accessed: 15 December 2025).

Data Storage

React Native Community (2024) AsyncStorage: An asynchronous, unencrypted, persistent, key-value storage system for React Native. Available at: https://react-native-async-storage.github.io/async-storage (Accessed: 15 December 2025).

Mobile Development Best Practices

Nielsen, J. (1994) '10 Usability Heuristics for User Interface Design', Nielsen Norman Group. Available at: https://www.nngroup.com/articles/ten-usability-heuristics (Accessed: 15 December 2025).

Google (2024) Material Design Guidelines. Available at: https://m3.material.io (Accessed: 15 December 2025).

Apple (2024) Human Interface Guidelines. Available at: https://developer.apple.com/design/human-interface-guidelines (Accessed: 15 December 2025).

Academic Sources on Mobile Development

CM3050 Mobile Development: University of London
coursera.org/learn/uol-cm3050-mobile-development/home/welcome

Eisenman, B. (2016) Learning React Native: Building Native Mobile Apps with JavaScript. 1st edn. Sebastopol: O'Reilly Media.

Masiello, E. and Friedmann, J. (2017) Mastering React Native. Birmingham: Packet Publishing.

Dabit, N. (2019) React Native in Action. New York: Manning Publications.